

WEEK 7 · LAB

Lab 3: Containerise an Application Properly

Anyone can write a Dockerfile that works. This lab is about writing one that is small, cached, reproducible and safe.

Prepared by **Dr. Abhijith Anandakrishnan**

Assistant Professor, Amrita School of AI

DURATION

One 2-hour block

SUBMIT

icclab3-<ROLLNO>.pdf

MARKS

12

OFFERING

Odd Semester 2026-27

The application

Use any small web application: a Flask or FastAPI service, an Express app, or a static site with a build step. It must have dependencies and a build step, or the lab teaches nothing.

Part 1: Make it work (2 marks)

1.1 Write a Dockerfile that builds and runs it. Record the image size.

1.2 Run it with a port mapping and confirm you can reach it.

```
docker build -t myapp:1.0 .  
docker run -d -p 8080:80 --name app myapp:1.0  
curl localhost:8080  
docker logs -f app
```

Part 2: Make the cache work (3 marks)

2.1 Time a rebuild after changing one line of source. Record it.

2.2 Reorder the Dockerfile so dependencies are installed **before** the source is copied. Time the same rebuild again.

2.3 Report both times and explain the mechanism in terms of layers and cache invalidation.

2.4 Add a `.dockerignore` excluding `.git`, `node_modules`, build output and any local environment file. Report the change in build-context size.

Part 3: Make it small (3 marks)

3.1 Convert to a **multi-stage build**: compile or install in a full image, copy only the artefacts into a slim runtime image.

3.2 Report image sizes for: your original, the multi-stage version, and a variant based on an `alpine` or `-slim` base.

3.3 Inspect the layers with `docker history`. Which layer is largest, and why?

3.4 Explain, with a number, why image size matters on a cluster that scales to fifty replicas.

Part 4: Make it safe and reproducible (3 marks)

4.1 Pin the base image to a specific version, not `latest`. Explain what `latest` breaks.

4.2 Add a non-root user and run as it. Verify with `docker exec app id`.

4.3 Pass a secret at **run** time through an environment variable or a mounted file, never baked into the image. Then prove a baked secret is recoverable: build a throwaway image with a secret in an `ENV` line and extract it with `docker history --no-trunc`. Delete that image afterwards.

4.4 Add a `HEALTHCHECK` and show it reporting healthy.

Part 5: State (1 mark)

5.1 Run a database container with **no** volume. Write data. Remove the container. Recreate it. Show the data is gone.

5.2 Repeat with a named volume. Show the data survives.

What to submit

icclab3-<ROLLN0>.pdf : both Dockerfiles, the rebuild timings table, the image size table, the docker history extract showing the recovered secret, and the volume demonstration.

MARKS	FOR
2	Part 1: it builds and runs
3	Part 2: cache ordering measured and explained
3	Part 3: multi-stage build with a real size reduction
3	Part 4: pinned base, non-root, secret handling demonstrated both ways
1	Part 5: the volume demonstration